

PROCESS & STAGES

The end-to-end remediation flow, from audit to validated Cloud data

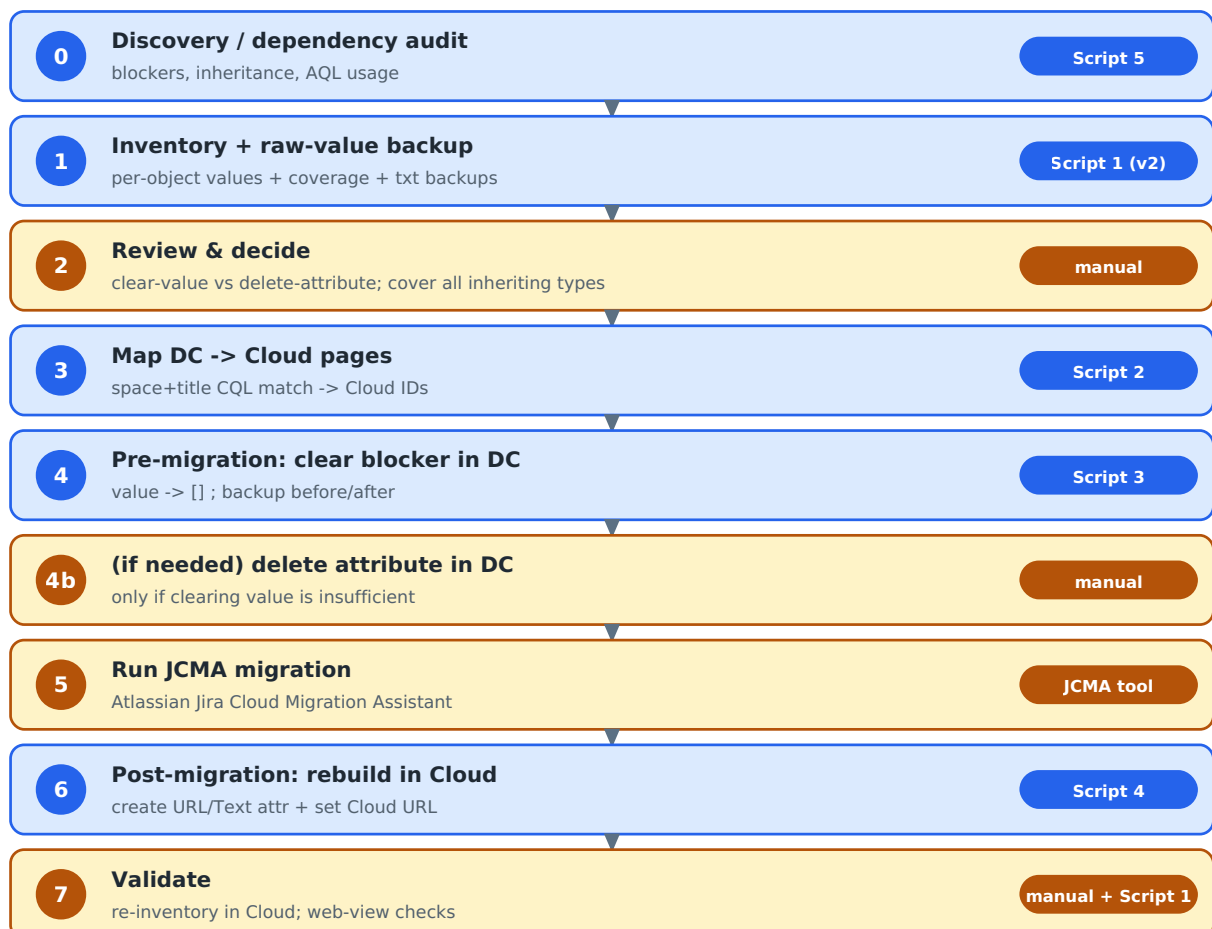
What happens in what order, which parts are automated by the scripts and which remain manual, and where each stage's output lives.

Assets DC→Cloud migration · attribute remediation series · document 2 of 4

Document set: (1) Data Model & Concepts · (2) Process & Stages · (3) Scripts Reference · (4) Runbook, Recovery & Rollback. Each part is standalone and can be regenerated independently.

1. The flow at a glance

Remediation runs in eight stages. The principle throughout: **never mutate the production DC source until the whole flow has been rehearsed on a sandbox**, keep the DC source as a clean rollback, and let backups plus dry-runs catch mistakes before they are irreversible.



2. What each stage does

Stage 0 — Discovery / dependency audit Script 5

Audits the schema(s) for *all* blocker attribute types (Confluence, Version, Project), detects inheritance, and scans for AQL usage. Produces the blocker landscape so nothing is missed. Output: an attribute audit CSV and a findings CSV. This is the map for everything that follows.

Stage 1 — Inventory + raw-value backup Script 1 (v2)

For each blocker attribute, lists every populated object and its value, parses out the link / space / page id, and writes one raw-value `.txt` backup per object. Output: a value CSV, a coverage CSV (which proves no populated object was missed), and a backup folder. This is your authoritative pre-change record.

Stage 2 — Review & decide manual

A human reads the audit and inventory and decides, per attribute: is clearing the *value* enough to unblock JCMA, or must the *attribute* be deleted? Confirm every inheriting object type is included. Confirm no AQL filter depends on the attribute. This is the one stage that genuinely needs judgement.

Stage 3 — Map DC→Cloud pages Script 2

Because Confluence page ids change on migration, this stage builds the bridge: for each DC link it finds the matching Cloud page by **space + title** via CQL and records the new Cloud URL and id. Can run any time before the post-migration rebuild (it only needs Confluence Cloud to exist). Output: a page-map CSV.

Stage 4 — Pre-migration: clear blocker in DC Script 3

Sets the blocker attribute's value to empty on each affected object, leaving the attribute definition in place. Dry-run by default; backs up the full object before and after for comparison; prints the object's web-view link so the result can be eyeballed. If clearing the value is *not* sufficient to satisfy JCMA, the attribute is deleted in the Assets UI instead manual — after backups exist.

Stage 5 — Run JCMA migration JCMA tool

The actual migration is performed with Atlassian's Jira Cloud Migration Assistant. With the blockers cleared, the Assets pre-check passes. This stage is outside the scripts.

Stage 6 — Post-migration: rebuild in Cloud Script 4

In the migrated Cloud schema, creates the replacement **URL** (or Text) attribute on the object type, then sets its value on each object to the mapped **Cloud** URL from stage 3. Staged, dry-run by default, with before/after backups of the object type and the object. Single-object test first, then bulk.

Stage 7 — Validate manual + Script 1

Re-run the inventory against Cloud to confirm the replacement attribute is populated where expected, spot-check objects in the Cloud web view, and reconcile counts against the original coverage CSV. Only then is the schema considered remediated.

3. Automated vs manual — coverage map

Activity	Covered by	Manual element
Find all blockers + dependencies	Script 5	Cross-check non-default types vs current JCMA matrix
Inventory values + back them up	Script 1 (v2)	Confirm coverage totals reconcile
Decide clear vs delete	—	Whole stage is a human decision
DC→Cloud page mapping	Script 2	Resolve NOT_FOUND / MULTIPLE rows by hand
Clear value in DC	Script 3	Deleting the attribute (if needed) is done in the UI
Run the migration	—	JCMA, entirely manual
Create attr + set values in Cloud	Script 4	Pre-create decision; verify in web view
Validate	Script 1 (re-run)	Spot-checks and sign-off

Not covered by scripts. Running JCMA itself; the clear-vs-delete decision; deleting an attribute in the UI when value-clearing is insufficient; and resolving page-map exceptions (NOT_FOUND / NO_TITLE / MULTIPLE). These are deliberate human checkpoints.

4. Where outputs live

Stage / script	Output
Script 5	assets_attribute_audit.csv , assets_blocker_findings.csv
Script 1 (v2)	confluence_attr_values.csv , confluence_coverage.csv , rawvalue_backup\
Script 2	confluence_pagemap.csv
Script 3	null_backup\<Key>_BEFORE/AFTER_*.json.txt
Script 4	cloud_backup\Cloud_*_BEFORE/AFTER_*.json.txt